

DepChain Report

Pedro Curto¹, Pedro Ribeiro¹, and Rodrigo Ganância¹

Instituto Superior Técnico, Portugal

pedro.a.curto@tecnico.ulisboa.pt, rodrigo.ganancia@tecnico.ulisboa.pt,
pedro.melo.ribeiro@tecnico.ulisboa.pt

Project Introduction

Goals

The project aims to develop a simplified permissioned blockchain system with high dependability guarantees (DepChain).

The initial delivery focuses on establishing secure and reliable communication between blockchain members and clients, implementing a Consensus protocol to ensure consistent and dependable state updates across all members of the system.

The second delivery focuses on allowing clients to conduct transactions on the blockchain using a native cryptocurrency and implementing smart contract execution, while ensuring fault tolerance even in the presence of Byzantine clients.

System Assumptions

The System assumes a static membership and a pre-established Public Key Infrastructure. The application can trust its local instance of the client library.

The blockchain operates on an unreliable and unsecured network, requiring the system to handle dropped and corrupted packets.

Additionally, some members and clients may be malicious and act arbitrarily. While the leader is assumed to be fixed and correct, the algorithms must be designed to tolerate potential Byzantine behavior (leader included) and allow for future extensions. Correct clients should wait to learn for the outcome of a request they submitted before submitting a new one - otherwise, in the case of transactions, the system will deny their transactions after a certain threshold.

Our system assumes that N , the number of members, is $3f + 1$, with f being the number of faults it tolerates.

Design Requirements

The implementation must always guarantee safety and ensure liveness only when the leader is correct. The system should remain functional as long as the number of Byzantine processes does not exceed $(n + f) / 2$.

A Byzantine process must not be able to add transactions that were never submitted by a client, cause inconsistencies where correct processes maintain divergent versions of the blockchain, enable invalid state transitions such double

spending, or, assuming the leader is correct, cause the system to stall. Clients and members can also collude, so the protocol must be resilient to coordinated attacks involving both faulty nodes and malicious clients.

Accounts and their balances should follow basic security and reliability rules. This includes making sure an account can't go below zero, only the right people can make changes to it, and ensuring non-repudiation for all operations on an account.

Implementation

Authenticated Perfect Links abstraction

Sessions

At startup, each blockchain member establishes a secure session with every other member. Clients initiate a session with all members for communication and to be able to get replies from all members.

To streamline connections, each member is responsible for initiating the protocol with members that have higher port numbers. Similarly, clients must initiate the protocol with all members.

The initiating entity generates a new AES key to be used as the session key and encrypts it using the recipient's public key. It then signs the message and transmits it to the other member.

Upon receiving the message, the recipient verifies the signature and decrypts the session key. Once established, all subsequent messages exchanged between these members must include an HMAC generated with the session key to ensure message integrity. This ensures that no member can impersonate another, since keys of honest clients or members cannot be subverted.

Reliable Transmission

Each entity maintains a sequence number for both sending and receiving messages during a session.

When sending a message, the sender appends its current sequence number to the message and waits for an acknowledgment (ACK) with the same number. If no ACK is received, the message is retransmitted at a fixed rate until the ACK is received, confirming successful delivery. Once the ACK is received, the sender increments its sequence number and notifies the application of successful message delivery.

Upon receiving a message, the receiver checks if the message's sequence number matches its expected receiving counter. If they match, the receiver sends an ACK and delivers the message to the application. If the message number is lower, it indicates the message has already been delivered, and the receiver only sends the ACK.

During connection establishment, both entities synchronize their sequence numbers to ensure consistency and a message is only accepted, even having the correct sequence number, if the HMAC is verified successfully.

Client Library

The client can perform two types of commands:

(1) requests that change the state of the blockchain, that will all be sent via a TRANSFER message from the clients to the members. These transfers can be in DepCoin (the native cryptocurrency of the blockchain) or in IstCoin (an ERC-20 token implemented through a smart contract) and must be interpreted accordingly by the members based on the associated transaction type. These can be:

- **Transfer** <address> <amount> (DepCoin, ISTCoin): Transfers an amount of tokens from our address to another
- **Transfer_From** <owner> <to> <amount> (ISTCoin): Transfers an amount from a given address to another via the allowance mechanism
- **Approve** <spender> <amount> (ISTCoin): Allows a given address to spend an amount of tokens from our address (allowance mechanism)
- **Blacklist** <address> (ISTCoin): Blacklists an address
- **Unblacklist** <address> (ISTCoin): Removes an address from the blacklist

and (2) requests that don't change the state of the blockchain and check state information:

- **Balance** <address> (DepCoin, ISTCoin): Checks the balance of an address
- **Allowance** <owner> <spender> (ISTCoin): Checks the allowance that a given address has provided to another
- **IsBlacklisted** <address> (ISTCoin): Checks if a given address is blacklisted

Consensus

Consensus is executed for requests that create modifications on the state of the system, including coin transactions (*approve*, *transfer*, *transferFrom* in case of ISTCoin), blacklisting addresses, or removing addresses from the blacklist grouped in a block. Other reading requests such as checking allowances, balances and blacklists don't require consensus execution and are broadcasted to every member by the client.

Nonces: Every client request uses a signed nonce to prevent replay attacks which all correct members verify before performing transactions.

Signatures: All requests are signed by the client. Signatures prevent tampering and correct members from deciding fake transactions proposed by a Byzantine leader. All Members verify the transaction signatures after the accepting phase and set invalid transactions as unsuccessful. Therefore, all correct members will not perform fake transactions, guaranteeing state agreement between correct members.

Starting consensus: The leader collects client's transaction requests for a period of time (defined as 5 seconds) before each consensus instance. Then the leader will aggregate all transactions and define an order in a block, in this case the leader sorts transactions by sequence number. A limit of transactions was defined per client (5 transactions) and per instance (15 transactions) to mitigate denial of service attacks, since the current implementation does not provide gas mechanism.

The Consensus algorithm follows the Byzantine Read/Write Epoch Consensus algorithm from the course book [1] (Algorithms 5.17 and 5.18).

The proposed value in each consensus instance is a block of transactions and a Byzantine Quorum corresponds to $N - f$, which is $2f + 1$ processes.

Similarly to the Algorithm from the book, it consists of two main phases, the **Read** and the **Write** phases.

Read Phase:

- The leader starts by setting its $\langle value, ts \rangle$ pair to the block of gathered transactions from clients requests, and the timestamp to zero. Afterwards, it broadcasts a **READ** to all members.
- The leader always waits for a specified **STATE_TIMEOUT**, to allow all correct processes to join in. During this time, each member receives the leader's **READ** request and sends its local **STATE** message, consisting of its $\langle value, ts \rangle$ pair, writeset, epoch, instance of the consensus, and signature over these contents. The instance of consensus is needed to prevent valid messages from previous instances to enter on following decisions, and the signature guarantees that when the leader sends the list of the states, the member can confirm the authenticity of each one.
- The leader collects all of these **STATE** messages and broadcasts them to all members, in the form of a **COLLECTED** message. At this point, all members, including the leader, will move to the *decideValue* function, that will decide what value each member chooses to write on the **WRITE** phase.
- After receiving the **COLLECTED** message, all processes ensure that they received at least a quorum of messages and check the signature of every **STATE** message inside it. Members select a value by choosing the $\langle value, ts \rangle$ pair with highest timestamp that appears in at least $N - f$ write sets. When counting the number of times in which it appears on the write set, the timestamp of the selected pair must be less than or equal to those of the same value present in the write sets. If no such pair exists, the default value is taken from the leader.

Write Phase:

- Each process broadcasts in a **WRITE** message, the value that it decided. It must wait for $N - f$ **WRITES** of the same value to **DECIDE**. Each process

only keeps a WRITE message from each process. Upon receiving the quorum of WRITES it broadcasts a DECIDE message. It then waits for $N - f$ DECIDE messages for the same value and delivers it to the blockchain, ending the Consensus instance. Every time a process is waiting for a quorum of messages, it timeouts after a period of time, aborting if it does not reach the quorum needed to advance, and moving on to the next epoch.

Reply Phase (additional to algorithm):

- After reaching consensus, appending the new block to the chain, and executing the transactions, the members respond to the clients to indicate whether their transfers succeeded or failed. To confirm the outcome of a transaction, a client must wait for matching responses from at least $f + 1$ members.

Genesis Block

Our system generates a genesis file in JSON format through a shell script. This file includes the smart contract details such as the contract's *address*, the *blacklist owner*, and the *bytecode* (both deployment and runtime). It also contains the hash of the genesis block, computed from the state information using a *Python* script, along with the previous block's hash and transactions (which are null and empty, respectively). For each client listed in the shell script, the file specifies their address, identifier, and balance in DepCoin. Client addresses are derived by hashing their public key and truncating it to the last 20 bytes, following a method inspired by Ethereum. All accounts, except for the owner, start with 0 ISTCoins, and their address remains the same for both DepCoin and ISTCoin transfers.

The genesis block contains the essential information that defines the initial state of the blockchain. On system startup, clients use it to get the account address with which they will perform requests, and information on what other client addresses exist. Members use it to extract key details - they initialize their local EVM instance, deploy the contract providing the owner and contract address and bytecode, and build the genesis block, initializing the *BlockchainState* with this initial block and the account information he extracted.

Appending and Persisting Blocks

Blocks are saved in JSON format after consensus is reached and all transactions within them are executed. They are hashed based on their content and on the previous block hash, ensuring that it is not possible to forge a false state.

To load the current blockchain state from JSON, it is necessary to apply the transaction history from the persisted blocks to the state present in the genesis block, and verify the hashes.

Transaction Execution

Transactions are processed in the order they appear in the block that was decided, ensuring consistent state across all members. Each transaction is validated - those with invalid signatures or incorrect nonces are rejected and marked as unsuccessful in the block. If they are correctly signed and have a valid nonce, the appropriate handler is called, either *ISTCoinHandler* for ISTCoin transactions or *DepCoinHandler* for DepCoin transactions. The result of executing such transaction is set as the status of the transaction, this means that if a client tries to transfer an amount of tokens larger than his balance, the execution of this transaction will fail but still remain in the block.

DepCoin: DepCoin is the native cryptocurrency of DepChain, which uses a straightforward state model consisting of accounts that map addresses to their corresponding balances. Each account is linked to a client through a known public/private key pair. Transactions update the state by directly modifying these balances as they are processed.

ISTCoin: ISTCoin is a fungible token that follows the ERC-20 standard. Its transactions are executed through smart contract functions. Before performing any *transfer*, *transferFrom*, or *approve* operation, the contract consults a blacklist to enforce access control.

Smart Contracts

Everything directly related with smart contract invocation is located in the **contract** package of our project. We implemented a *ISTCoin.sol* contract that merges both requirements for an ERC-20 fungible token and a blacklist-based access control list. This implementation was done in Remix and extends the *OpenZeppelin's* ERC20 implementation.

Both the deployment and runtime bytecode as well as the method identifiers were extracted and placed in a **ContractFunctions** class. In this class, we placed all of the methods that lead to smart contract execution. It contains a method to deploy the contract, methods to call basic information such as checking the name, symbol, total supply and decimals, and to trigger *balanceOf*, *allowance*, *transfer*, *transferFrom*, *approve*, *blacklist*, *unblacklist* and *isBlacklisted*.

Each member is initialized with an instance of an *EVMExecutor* for contract deployment and a *ByteArrayOutputStream*. After consensus is reached and the necessary checks are completed, all transactions in ISTCoin are processed by an *ISTCoinHandler*, which invokes the relevant functions in *ContractFunctions*. If the client's signature is valid and the request is authorized, it triggers the execution of the smart contract. Thanks to the consensus process, we ensure that correct processes execute the same set of transactions in the same order, maintaining consistency.

For the blacklist, we have designated a global owner, specified in the genesis file, who has the authority to modify its state. The blacklist restricts the *Transfer*, *TransferFrom*, and *Approve* operations. For the operation to succeed, none of the involved parties (sender, recipient, or spender, if applicable) can be blacklisted.

Tests

Most of the automated and relevant tests (using JUnit) are located within the *member* module. Additionally, there are tests in the *contract* package to verify the functionality of the smart contract when invoked through Java code (Step 2). These tests check whether expected outputs are returned, whether blacklisting behaves correctly, and whether token transfers and the allowance mechanism function as intended.

There is also the possibility of running your own manual tests, using the appropriate shell scripts to launch either all members as non Byzantine, or a Byzantine member with configurable behaviour - the way of accomplishing this or the tests below is described in our *README.md*. Byzantine clients also exist, but they are only useful in the tests, so the user should not run them manually.

Regular Scenario

A simple test was included to show the behaviour of the program where all clients and members are non-byzantine. All clients perform transactions from one to another, and the allowance mechanism (approve and transferFrom) is also tested.

Simulating Byzantine Behaviour

To cope with possible Byzantine behaviour from members and clients, we devised a series of scenarios that corroborate the correct functioning of the program with these adversarial conditions. Starting with the **member tests**:

Ignoring Messages: A subset of f processes ignore all messages, essentially becoming useless members. Since the system tolerates this failure, it continues processing transactions and appending them to the Blockchain, and the system isn't disrupted.

Spam Messages: A subset of f processes spam the leader with a configurable number of messages - these include **STATE**, **WRITE** and **ACCEPT** messages. The system continues functioning normally.

Impersonation: The byzantine process picks another member at random, and attempts to send the message as if it was him. The test asserts that, despite this attempt, the leader notices that an invalid signature was detected, doesn't

consider that **STATE**, and the system continues to exhibit normal behaviour.

Incorrect States: The byzantine processes try to propose a value to the consensus with a higher timestamp than it was supposed to, exploiting the fact that the other correct processes would be tempted to pick it and then send broadcast **WRITE** and **ACCEPT** messages for it. However, since $n - f$ did not propose this value, this attempt fails and the system continues working correctly.

Wrong Writes and Accepts: Finally, a subset of byzantine processes send forged **WRITE** and **ACCEPT** messages, but fail to disrupt the system.

The tests that we devised for the **byzantine clients** were:

Fake Signature: A Byzantine client sends a transaction request along with a fake signature to the members. We assert that the response back to the client is negative, confirming to him that his transaction request did not succeed.

Replay Attack: A Byzantine client waits for a block to be decided in a first instance of consensus. He then selects a valid transaction that was already executed, constructs a request and attempts to replay this transaction. Because the members track client nonces, they promptly flag this as a replay attack and reject his attempt.

Conclusion

This project demonstrates the successful implementation of a dependable and secure blockchain platform, featuring native cryptocurrency support through DepCoin and smart contract execution via the ERC-20 compliant ISTCoin. Through careful design of consensus mechanisms, access control (including blacklist enforcement), and state consistency, the system ensures fault tolerance in the presence of Byzantine clients and members.

The genesis block establishes a clear and verifiable initial state, while transactions processed in a deterministic order across all correct members maintain integrity and consistency. Automated testing, primarily using JUnit, has validated the correctness of core functionalities, including transaction execution, smart contract behavior, and blacklist enforcement.

Overall, this delivery reflects a solid step toward a reliable blockchain infrastructure with strong guarantees for correctness, consistency, and security. It lays a robust foundation for future enhancements, such as more complex smart contracts, richer permissioning models, and broader client interactions.

References

1. Christian Cachin, Rachid Guerraoui, and Luís Rodrigues. *Introduction to Reliable and Secure Distributed Programming*. 2nd Ed. Springer, 2011.